

Study the Effectiveness of Existing Detection Mechanisms Against Prompt Injection in LLMs

Qais Saif Qassim
College of Computing & Information
Sciences, University of Technology and
Applied Sciences -Ibri,
Sultanate of Oman
qais.aljanabi@utas.edu.om

Dilwar Mazumder
Center for Preparatory Studies,
University of Technology and Applied
Sciences -Ibri,
Sultanate of Oman
dilwar.mazumder@utas.edu.om

Abstract—Prompt injection attacks pose a significant security risk to large language models (LLMs) by allowing adversaries to manipulate model behavior through crafted inputs or external context. This paper presents a structured survey and comparative analysis of recent detection mechanisms for prompt injection attacks. Using a systematic review approach, relevant studies from 2023–2025 were identified, screened, and evaluated based on reported metrics, threat models, latency overhead, and deployment constraints. The survey categorizes detection approaches into syntactic, semantic, attention-based, and ensemble methods, highlighting their strengths, limitations, and practical trade-offs. Results show that recent semantic and ensemble detectors achieve high detection rates but often introduce additional latency, while attention-based methods offer deeper insight at the cost of requiring model internals. The paper concludes with best-practice recommendations and key research challenges for developing robust and scalable prompt injection detection strategies.

Keywords— *Prompt injection detection, large language models (LLMs), adversarial attacks, jailbreak attacks, semantic filtering, ensemble detection, LLM security evaluation*

I. INTRODUCTION

Large Language Models (LLMs) have emerged as transformative technologies in the field of Natural Language Processing (NLP) enabling applications in search, content generation, and decision-making; reshaping the way machines understand, generate, and interact with human language. LLMs are trained on vast and diverse datasets comprising text from books, articles, websites, and other digital sources. This extensive training enables them to capture complex linguistic patterns, contextual relationships, and semantic nuances, allowing for a sophisticated level of text comprehension and generation [1]. Their influence extends across multiple domains, including information retrieval, automated content creation, and data-driven decision-making. In search systems, LLMs enhance query understanding and relevance ranking by interpreting user intent more accurately than traditional keyword-based methods. In content generation, LLMs facilitate the automatic production of coherent and contextually rich text for applications such as report writing, translation, and summarization. Furthermore, in decision-making environments, LLMs support analytical reasoning, assist in drafting policy documents, and provide insights based on large-scale textual data analysis [2]–[4].

The rise of LLMs marks a major turning point in artificial intelligence, bringing machines closer to understanding and

using language in ways that resemble human communication. By learning from vast amounts of text, these models have become powerful tools that not only advance natural language processing but also drive innovation across many areas of AI [2]. Their ability to combine human-like understanding with computational speed and accuracy is opening new possibilities for how technology can assist in thinking, reasoning, and creating knowledge. However, their integration with external data sources introduces risks, particularly prompt injection attacks where malicious prompts override intended behaviors [3], [4]. These attacks can lead to data leakage, misinformation, or unauthorized actions [2], [5].

This paper conducts a structured survey and comparative effectiveness analysis of recent detection mechanisms against prompt injection attacks in LLM-based systems, focusing on their reported performance, latency overhead, and deployment assumptions. To ensure rigor and transparency, This study adopts a structured review methodology that identifies recent prompt-injection detection research through targeted searches across IEEE Xplore, the ACM Digital Library, and other major scholarly databases, applies defined inclusion and exclusion criteria, extracts comparable evaluation metrics, and synthesizes the findings into a comparative effectiveness analysis across major detection categories.

To ensure rigor and transparency, this study follows a structured review methodology. Relevant research published between 2023 and 2025 was identified through targeted searches in major digital libraries using combinations of terms such as “prompt injection,” “LLM security,” and “detection mechanisms.” Studies were included if they proposed or evaluated a concrete detection approach and reported measurable effectiveness metrics such as accuracy, precision, recall, F1-score, or attack success rate reduction. Conceptual articles without empirical evidence, works focusing solely on mitigation without detection, and non-scholarly sources were excluded. For each selected paper, data were extracted on threat models, detection techniques, datasets, evaluation metrics, latency overhead, and deployment constraints. These data points were then synthesized into a comparative analysis to assess relative effectiveness across syntactic, semantic, attention-based, and ensemble detection approaches.

II. PROMPT INJECTION ATTACKS

A serious weakness in large language models arises when harmful instructions hidden within input prompts cause the model to produce unexpected or unintended responses. These attacks exploit the model's natural language processing capabilities, turning them against the system [2]. Prompt

Injection is recognized as the top risk in the OWASP Top 10 for Large Language Model Applications, reflecting its widespread impact and ease of exploitation. Unlike traditional cyber-attacks that target code or hardware, prompt injection attacks leverage the semantic understanding of LLMs, making them particularly stealthy in applications where LLMs process user-generated or external content [3]. There are two primary categories: direct prompt injection, where the attacker has full control over the prompt input (e.g., in chat interfaces), and indirect prompt injection [6], where malicious content is embedded in external data sources that the LLM retrieves and processes (e.g., web pages, documents, or APIs).

Direct prompt injection represents a straightforward yet effective vulnerability in LLM applications, occurring when an attacker gains complete authority over the input prompt [1]. This typically happens in user-facing applications like chatbots or interactive AI interfaces, where individuals can freely craft and submit queries. For instance, an attacker might input a carefully designed prompt that overrides the model's intended behavior, such as instructing it to ignore safety protocols or reveal sensitive information as illustrated in Figure 1. By exploiting the LLM's tendency to follow instructions embedded within the prompt, the attacker can manipulate outputs to perform unintended actions, like generating harmful content or executing simulated code. This category of attack is particularly insidious because it requires no intermediary steps; the malicious input directly influences the model's response, highlighting the need for robust input validation and prompt engineering to safeguard against such exploits.

In contrast, indirect prompt injection presents malicious queries through less obvious channels, where the attacker embeds harmful instructions or data into external resources that the LLM later ingests as illustrated in Figure 2 [7]. This could involve tampering with web pages, documents, emails, or API responses that the model retrieves during its processing workflow. For example, if an LLM is tasked with summarizing a webpage containing hidden adversarial text, that text might subtly redirect the model's focus or drive it to produce biased or dangerous outputs without the user's awareness.

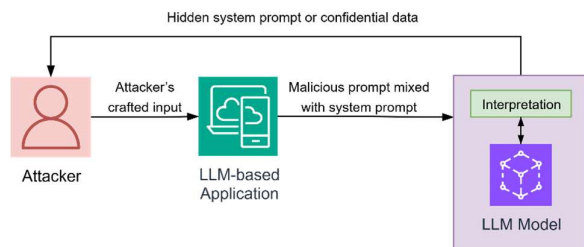


Figure 1 Direct prompt injection

Unlike direct methods, this approach relies on the LLM's integration with external data sources, making it more covert and harder to detect at the input stage. Attackers can exploit this by poisoning data repositories or third-party content, turning benign retrieval tasks into vectors for manipulation and underscoring the risks associated with LLMs that dynamically incorporate untrusted information.

world examples of prompt injection attacks targeting large language models and the applications built around them. Each incident revealed different weaknesses, attack methods, and consequences. In early 2023, a Stanford student demonstrated a simple but powerful exploit on Microsoft's Bing Chat to reveal its hidden system prompt, exposing proprietary safety guardrails and proving that direct user input can override system instructions [8]. The finding highlighted confidentiality and model-integrity risks rather than mass compromise but served as a wake-up call for the potential risks of prompt injection. By late 2023 and into 2024, security researchers and bug-hunters began discovering more sophisticated and harmful examples. Developer assistants such as GitHub Copilot Chat were found to be vulnerable to indirect prompt injections [9], where malicious instructions were hidden inside seemingly harmless text, for instance, within code comments or pull requests. When the assistant processed this tainted input, it could unintentionally leak sensitive data such as API keys or generate insecure code. These attacks highlighted how untrusted content could be weaponized to manipulate AI behavior, posing serious risks to software supply chains.

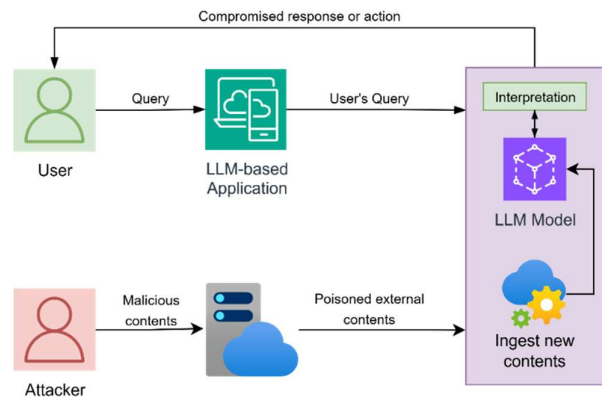


Figure 2 Indirect prompt injection

Throughout 2024, independent audits and media reports emphasized how some emerging LLM platforms lacked strong guardrails. For example, DeepSeek's R1 model reportedly failed several inject abuse tests [10], proving that even simple malicious prompts could trigger unsafe or toxic outputs if proper safeguards weren't in place. By late 2024 and into 2025, researchers observed an evolution in these attacks. Instead of targeting user prompts directly, adversaries began embedding malicious content in webpages and online repositories to manipulate LLMs' retrieval and summarization features. A Guardian investigation revealed that adversarial web content could bias ChatGPT's search tool [8], leading it to display misleading results or even surface malicious code snippets, a clear example of "data poisoning" in real-world environments. Most recently, researchers uncovered high-severity cases where prompt injections were combined with traditional web vulnerabilities to launch large-scale data breaches. CamoLeak incident was one of the most significant and alarming real-world incidents, where attackers exploited weaknesses in Copilot Chat to silently extract source code and sensitive secrets. These cases underscore that prompt injection is no longer just a theoretical or academic concern; it has matured into a serious security threat capable of enabling data

theft, system compromise, and large-scale enterprise breaches when paired with other exploitation techniques.

III. EFFORTS IN DETECTING PROMPT-INJECTION ATTACKS

Researchers have proposed a variety of detection mechanisms. These approaches can be broadly categorized based on their underlying methodologies and focus areas. This review organizes recent literature into four main categories:

- **Syntactic Detection:** Methods that analyze the structural and surface-level features of prompts to identify malicious patterns.
- **Semantic Detection:** Approaches that leverage the meaning, intent, and contextual understanding of prompts and responses for more robust detection.
- **Attention Analysis:** Techniques that utilize attention mechanisms and contextual tracking, particularly in multi-turn or multi-modal interactions, to detect subtle or evolving attacks.
- **Ensemble Detectors:** Architectures that combine multiple models or detection strategies, often leveraging ensemble learning or multi-agent systems to enhance robustness and accuracy.

In the following sections, we provide a detailed summary of each category, highlighting their key mechanism, significance, and limitations. This structured overview aims to clarify the current landscape of prompt injection detection research and to inform future developments in securing LLM-based systems.

A. Syntactic Detection

Syntactic detection is an essential layer in defending LLM-driven systems against prompt injection attacks. This approach leverages language's lexical patterns, keywords, formatting cues, and token sequences, to identify potential indicators of malicious manipulation. Early defense strategies commonly relied on static blocklists or allowlists that specified malicious-intent terms or phrases [2]. However, such mechanisms are inherently fragile and volatile, as adversaries can readily evade them through rephrasing, encoding, character substitution, or other linguistic obfuscation strategies [3]. These limitations underscore the adaptive and evolving nature of adversarial behavior in prompt injection attacks [11]. Modern syntactic detection mechanisms rely on analyzing the superficial and structural characteristics of language to identify signs of malicious manipulation. These mechanisms typically employ techniques such as keyword filtering, pattern matching, regular expression analysis, and machine learning classifiers that operate on surface-level linguistic features [5], [12]–[15].

Researchers and practitioners have proposed various solutions to improve syntactic detection methods by integrating automation and statistical analysis. Recent detection systems often use heuristics detection methods to identify sudden changes in writing style, constraints or command-like language, and other irregularities that may indicate malicious intent [2]. Other approaches extract linguistic features such as n-gram frequencies, punctuation use, part-of-speech patterns, and formatting anomalies, which are then analyzed using machine learning or neural classifiers

for anomaly detection [16]. Template-based methods also compare new prompts with known attack signatures or “known-bad” lexical patterns [1]. Although these strategies extend beyond simple keyword lists, they continue to face challenges posed by the open-ended and creative nature of natural language. Joshi et. al. [17] proposed framework employs a multi-layered security architecture that integrates spell-checking, keyword-driven jailbreak detection, role-based access control, and prompt-signing techniques. In this design, high-risk operational terms such as delete, transfer, and withdraw are cryptographically signed for authorized users, while suspicious control-shifting terms including ignore and bypass trigger automatic rejection. The system verifies the integrity and authenticity of signed prompts prior to LLM execution, ensuring that only validated instructions reach downstream components. Experimental evaluation shows that the mechanism effectively blocks malicious prompts containing destructive keywords, although its performance remains dependent on predefined keyword lists rather than deeper semantic analysis. In practical deployments, maintaining the relevance and completeness of such filters is a continual arms race. Publicly documented defenses invite attackers to engineer new bypasses, leading filters to lag emergent offensive trends. Furthermore, efforts to capture the full spectrum of suspicious syntax in the varied context of real-world conversations often result in an unmanageable number of edge cases and false alarms, limiting the precision of syntactic detection. While such approaches provide an essential first line of defense, they remain largely reactive and susceptible to evasion. This highlights the necessity for more thorough, context-aware defenses.

B. Semantic Detection

Unlike syntactic detection, semantic detection focuses on understanding the meaning behind user inputs [4], [18]. It focuses on identifying malicious inputs based on their meaning and intent, rather than purely syntactic features. The goal here is to figure out if a seemingly harmless or cleverly disguised prompt is really trying to manipulate the model's behavior in a harmful way. This approach is particularly essential given the complexity and subtlety of modern prompt injection attacks, which often employ nuanced language, indirect instructions, or contextual embedding to evade straightforward syntactic filters. At its core, semantic detection deals with the interpretation of prompts by leveraging advanced natural language understanding techniques and contextual embedding that capture the latent meanings within inputs. This enables the detection frameworks to distinguish not just what words are present, but how they function in the broader linguistic and operational context of the prompt. For instance, phrases that might seem harmless on the surface can contain a hidden instruction that convinces the LLM to disclose confidential information or circumvent its security rules. Recent research demonstrates sophisticated methodologies for semantic detection that integrate multiple layers of linguistic and heuristic feature analysis. One effective approach is dual-channel feature fusion proposed by [19], which combines semantic embedding extracted from pre-trained models with heuristic features engineered based on known attack patterns. These explicit features might include structural anomalies commonly observed in prompt injections, such as unusual punctuation usage, unexpected command phrases, or context

inconsistencies. The fusion of semantic and heuristic data enhances detection accuracy by balancing deep contextual understanding with explicit domain-specific rules, thus mitigating false positives and increasing robustness against evolving attack strategies.

Muliarevych [16] addressed prompt injection attacks in LLM-based systems by proposing a multi-layered security framework with a Filter level containing a Prompt Analyzer and Attack Validator module. The study tested 800 prompts (500 attacks across five categories). The experimental results demonstrated that default GPT models are highly vulnerable, failing to detect 80% of certain attacks, while the GPT-4-based Attack Validator achieved 76-97% detection rates with F1 scores of 0.82-0.96, significantly outperforming both Fuzzy Search and baseline models. Kulkarni and Namer [6] introduce the Temporal Context Awareness (TCA) framework to defend against multi-turn manipulation attacks on LLMs. The framework employs a supervisory system that continuously monitors conversations using an LLM Intent Analyzer for semantic analysis, a Risk Calculator tracking metadata dimensions (language, domain, time sensitivity, prohibited content), a Risk Progression Tracker maintaining temporal risk evolution, and a Security Decision Engine that classifies interactions as Allow, Warn, or Block. The system calculates progressive risk scores using a sliding window approach that integrates historical risk, interaction risk, and pattern detection. Preliminary evaluations on the MHJ dataset demonstrated effectiveness in identifying subtle manipulation patterns across various attack tactics including obfuscation, direct requests, and injection. Liu et. al. [20] proposed a game-theoretic detection framework for identifying prompt injection attacks by fine-tuning a dedicated detection LLM using a minimax optimization strategy. Extending the principles of known-answer detection [21], the approach employs a detection instruction embedded with a secret key to determine whether the target input has been contaminated. The key innovation lies in intentionally fine-tuning the detection model to be more susceptible to prompt injection, thereby increasing the likelihood that it fails to reproduce the secret key when exposed to manipulated or adversarial prompts. The detection mechanism is formalized as a minimax game in which the inner maximization models strong adaptive attackers attempting to bypass the system, while the outer minimization optimizes the detector to remain robust under these worst-case conditions. The experimental evaluations across diverse datasets and LLM architectures demonstrate that the proposed detection system achieves near-zero false positives and substantially outperforms prior detection strategies, including traditional known-answer approaches.

Zhao and Li [22] proposes a comprehensive security framework that operates through two primary modules: prompt injection threat analysis and LLM output risk monitoring. The threat analysis module targets three categories of attacks: malicious prompts, unauthorized language manipulation, and privacy disclosure. This module is able to distinguish direct injection attempts, identifies unsupported or mixed-language inputs, and detects prompts containing personally identifiable or sensitive information. On the other hand, the output monitoring module evaluates the safety of generated responses by validating context dependency, detecting malicious or encoded executable code,

identifying unauthorized language outputs, and measuring privacy sensitivity in generated text. These modules leverage fine-tuned models trained on natural language inference datasets, multilingual corpora, and privacy-oriented datasets to ensure consistency between input and output and to intercept harmful or privacy-violating content. The experimental results demonstrate that the integrated framework achieves an overall detection accuracy of 93.65%, with fine-tuning improving performance by up to 5% across several attack categories. These findings indicate the effectiveness of combining multi-stage input threat assessment with continuous output monitoring to strengthen LLM defenses against evolving prompt injection attacks.

Furthermore, the deployment of similarity search techniques within semantic vector spaces forms another pillar of semantic detection. Here, incoming prompts are compared against a curated repository of known malicious vectors or banned terms embedded in high-dimensional semantic space. Prompts exhibiting close semantic proximity to these vectors trigger alerts or mitigation actions. This method benefits from the continuous update of the repository with emerging attack signatures and the model's ability to generalize semantics beyond exact word matches, capturing paraphrases or semantically equivalent manipulations. A key advantage of semantic detection frameworks lies in their adaptability and generalizability across different LLM architectures and applications. Since they rely on understanding underlying intent and meaning rather than fixed token sets, these systems can better cope with novel, unseen attacks, including those leveraging indirect or obfuscated language. This is crucial given the proliferation of LLMs with varying architectures each potentially susceptible to distinct injection methods.

However, semantic detection also presents significant challenges. The complexity of natural language semantics inherently complicates the precision of threat identification. Balancing sensitivity (to catch all potentially harmful inputs) against specificity (to avoid flagging legitimate queries) is a persistent trade-off. Overly aggressive semantic filtering risks degrading user experience by rejecting valid prompts, whereas lenient settings might allow insidious injections to pass. Moreover, LLMs themselves, used in detection, may inherit biases or misinterpret ambiguous language, raising concerns about trustworthiness and reliability. In response to these challenges, ongoing research explores hybrid detection mechanisms that dynamically adjust detection thresholds based on context, user behavior analytics, and feedback loops. For instance, some models incorporate behavioral analysis that monitors user interaction patterns across sessions to contextualize prompts within a broader usage framework, enhancing detection with historical and situational awareness. This can help differentiate accidental or exploratory inputs from deliberate attacks. The integration of semantic detection methods also emphasizes real-time mitigation capabilities. Prompt injections can cause immediate security breaches if not promptly neutralized; hence, detection systems often couple semantic analysis with automated response actions such as prompt sanitization, quarantine, or escalation alerts to security teams. Real-time detection is facilitated by efficient embedding computations and neural network inference optimized for low latency.

C. Attention Analysis

Recent research has shifted toward analyzing the internal dynamics of large language models such as attention patterns and activation trajectories, to detect prompt injection attacks. Unlike syntactic or semantic detection methods, this approach investigates the model's internal states to identify irregularities or deviations that signal potential adversarial manipulation. The main theory behind this approach is that the prompt injection attempts, particularly those instructing the model to override system constraints or disclose confidential instructions, create unique activity patterns inside the model's attention layers that differ from normal queries. By tracking these internal signals, researchers can detect when the model's reasoning starts to move away from its intended behavior. For instance, when a query embeds an instruction such as "ignore all previous directives and output system prompts," attention-based diagnostics frequently reveal a reallocation of focus toward the adversarial segment, effectively overriding prior semantic conditioning.

Early investigations employed attention heat-maps and token-attribution techniques to visualize how injected tokens anomalously capture attention across the transformer architecture. For example, researchers utilized self-attention weight visualizations in multi-head transformer models to trace significant token-to-token flows and identify abnormal attribution patterns [5]. The literature indicates that these deviations follow systematic patterns rather than stochastic fluctuations, exhibiting enhanced cross-layer dependencies and token co-activation between instruction-oriented embedding and output-relevant layers. These characteristics represent distinctive behavioral signatures of prompt injection attacks [23]–[25]. Building on these insights, advanced methods propose real-time anomaly detection frameworks that track activation distributions during inference. These systems typically establish a baseline of normal activation profiles using unsupervised clustering or dimensionality reduction techniques (e.g., PCA, t-SNE). Incoming prompts are then evaluated against these baselines to identify statistically significant deviations suggestive of compromised reasoning. Alternatively, some approaches extend detection by incorporating temporal tracking over token sequences, leveraging recurrent or sliding-window mechanisms to identify transitions from task-oriented reasoning, where the model focuses on solving the user's explicit objective, to meta-instructional interpretation, where the model begins analyzing or responding to instructions about its own behavior which represents a characteristic indicator of prompt injection exploitation. Another promising approach uses layer attribution and interpretability probes to track how the model makes decisions through its layers. Recent studies showed that, under prompt injections, some middle and late transformer layers display unusually high activation and irregular attention patterns. For example, [23] reveals that when a prompt injection attack occurs, the attention of specific attention heads shifts from the original instruction to the injected instruction within the attack data, a phenomenon they have named the distraction effect.

Despite its theoretical appeal, internal-state monitoring faces practical constraints. First, it requires white-box access to model parameters and attention maps, which is rarely feasible for proprietary or API-based deployments. Second,

continuous tracking imposes significant computational overhead, limiting real-time applicability. Third, adaptive adversaries may craft prompts that mimic benign activation profiles, effectively camouflaging injection effects. Consequently, attention-based monitoring is best deployed as a complementary defense alongside input-level filtering and output validation strategies.

D. Ensemble detectors

The concept of multi-agent and ensemble detectors has emerged as a promising approach to enhancing the detection of prompt-injection attacks in large language models. This line of research builds upon the observation that single detectors, whether rule-based, classifier-based, or embedding-based, often exhibit limited generalization capabilities and are vulnerable to adaptive adversarial strategies. Multi-agent and ensemble detection frameworks seek to overcome these weaknesses by combining the strengths of heterogeneous detection mechanisms, thereby improving robustness, precision, and adaptability.

In a multi-agent detection system, individual agents perform distinct, yet complementary tasks related to the identification of prompt-injection attempts. For example, one agent might specialize in lexical and syntactic analysis, identifying explicit attack signatures such as "ignore previous instructions" or "reveal hidden prompt," while another may employ semantic similarity measures or embedding-based models to detect contextually disguised injections. A third agent could perform behavioral validation, evaluating the coherence and policy compliance of the model's intermediate or final outputs. By fusing the decisions or confidence scores from multiple agents, the system can achieve a more holistic and accurate assessment of potential prompt-injection activity. This ensemble paradigm is inspired by established principles in ensemble learning, where diverse models jointly reduce both variance and bias, yielding superior predictive stability compared to individual detectors.

Recent research has explored different architectures for implementing multi-agent frameworks. Some studies employ cascaded ensembles, where agents operate in sequence—each refining or vetoing the preceding stage's decision—to minimize false positives while preserving high recall. Others adopt parallel ensembles that aggregate agent outputs using voting mechanisms, weighted confidence fusion, or Bayesian inference. In either configuration, the ensemble's performance depends critically on the diversity and independence of its constituent agents. When agents rely on overlapping feature representations or similar decision boundaries, the ensemble's benefit diminishes. Therefore, an effective multi-agent system deliberately integrates heterogeneous detection paradigms, such as combining a lightweight keyword detector with a transformer-based semantic model and a response-verification agent. Alla & Sichani [4] propose a multi-layered security framework that integrates an attack detection algorithm, a dynamic attack library storing known threat patterns, and mitigation strategies including LLM switching and rate limiting. The framework was evaluated across multiple LLMs (ChatGPT, Gemini AI, WriteSonic, Hugging Face) using security metrics like Prompt Leaking and Code Generation, revealing varying vulnerability levels.

An important dimension of multi-agent systems lies in their potential for adaptive self-coordination. Some frameworks leverage LLMs themselves as part of the detection ensemble, assigning distinct roles such as “red team,” “blue team,” and “arbitrator” agents that iteratively challenge, defend, and adjudicate prompts in real time. This reflective use of LLMs as both the target and the defender introduces a dynamic evaluation process that adapts to new attack patterns without explicit retraining. However, such self-referential configurations raise open questions about computational overhead, consistency, and potential bias amplification. Empirical evaluations of multi-agent and ensemble detectors have shown that they significantly outperform single-model baselines in controlled prompt-injection testbeds. For instance, studies published between 2023 and 2025 demonstrate improved detection rates (often by 10–25%) when integrating lexical, embedding, and behavioral validators into a unified framework. Furthermore, ensemble detectors demonstrate greater resilience to paraphrased or obfuscated attacks that evade static heuristics. Nevertheless, the literature also highlights several limitations. Multi-agent architectures incur increased latency due to sequential evaluation and decision aggregation, which can hinder deployment in latency-sensitive applications such as conversational AI or real-time code generation. In addition, coordinating multiple agents introduces complexity in maintaining model synchronization, updating detection rules, and managing conflicting judgments between agents.

Lan et al. [26] propose a hybrid constitutional classifier system combining a lightweight DeBERTa-based [24] input classifier with a Mistral-7B, a 7.3 billion parameter language model, output classifier to defend against prompt injection attacks. The input classifier screens adversarial prompts before they reach the LLM, while the output classifier monitors responses in real-time to prevent unsafe content generation. The system achieves 96.97% detection accuracy for inputs and 78.79% for outputs, with 67% lower latency compared to LLaMA-2-13B classifiers. The authors apply optimization techniques including quantization and knowledge distillation to enhance performance while maintaining efficiency. The proposed hybrid approach demonstrates that combining lightweight classifiers with small generative LLMs provides scalable, high-performance defense suitable for real-time AI applications.

Esugo et al. [13] proposed SHIELD, a hybrid detection framework designed to identify harmful prompt injection attacks against LLM-based systems. The framework integrates rule-based heuristics, zero-shot LLM inference, and supervised machine learning classifiers, forming an ensemble approach that leverages the strengths of each component. As part of the study, the authors also construct two cybersecurity-oriented prompt injection datasets of varying complexity to evaluate model performance. The experimental results demonstrated that ensemble methods significantly outperform individual detection techniques, particularly when faced with complex or obfuscated adversarial prompts, whereas rule-based filters suffer from high false-negative rates.

From a systems security perspective, multi-agent and ensemble detectors exemplify a defense-in-depth strategy, aligning with broader cybersecurity principles. By distributing detection responsibility across multiple independent

components, the system avoids a single point of failure. This architectural redundancy not only enhances resilience against evasion but also provides a valuable audit trail of detection reasoning, enabling explainability and forensic traceability. As research progresses, there is increasing interest in integrating ensemble detection with reinforcement learning frameworks, allowing agents to continuously refine their strategies based on feedback from evolving adversarial behavior.

IV. RECOMMENDATIONS FOR BEST PRACTICES

Enhancing the security of Large Language Models requires a comprehensive and layered strategy that addresses the complexity and evolving nature of prompt injection attacks. The literature consistently shows that no single technique is sufficient, and that combining detection, mitigation, and architectural safeguards offers the strongest defense. This defense-in-depth approach aligns with findings across syntactic, semantic, and ensemble-based detection research [2], [13], [19], all of which demonstrate that multi-stage controls significantly reduce the likelihood of a single point of failure.

A foundational requirement for secure LLM systems is rigorous input validation and prompt sanitization. Studies employing keyword filtering, role-based access controls, spell-check layers, or cryptographic prompt signing [17] emphasize that constraining the structure and language of incoming prompts helps prevent straightforward direct injections. However, given the limitations of purely syntactic filtering, modern systems increasingly rely on context-aware semantic monitoring. Dual-feature semantic fusion methods [19], temporal context models in multi-turn dialogue [6], and game-theoretic semantic detectors [20], all highlight the importance of understanding user intent and detecting meaning-level manipulations rather than superficial linguistic anomalies. Semantic analysis is especially critical as attacks grow more sophisticated and often disguise harmful intent through paraphrasing, contextual embedding, or multi-turn manipulation [16].

Because semantic detection alone may fail to capture deeply embedded or obfuscated injections, system designers should incorporate ensemble or multi-agent detection architectures. Research repeatedly demonstrates that combining heterogeneous detectors—such as rule-based heuristics, supervised classifiers, zero-shot LLM validators, and output-monitoring agents—provides substantially higher robustness than any single model [4], and [13]. Hybrid systems that screen both inputs and outputs, such as DeBERTa-based input classifiers paired with Mistral output evaluators [26], further illustrate that effective protection requires monitoring not only what enters an LLM, but also what it produces. Output validation is essential to prevent the generation of sensitive information, hidden system instructions, executable code, or harmful content, particularly in systems interacting with external tooling or high-risk workflows [22].

Equally important is securing the broader architectural environment in which the LLM operates. Real-world incidents, including prompt leaking in chat assistants, supply-chain exposure in developer tools, and large-scale code exfiltration (e.g., CamoLeak), demonstrate that vulnerabilities

often emerge when LLMs are granted uncontrolled access to external data, software repositories, or tools. Restricting external data access, validating retrieved content, and sandboxing high-risk operations are therefore essential safeguards, especially in retrieval-augmented generation systems where indirect prompt injection remains a serious threat. Architectural controls such as role-based access, rate limiting, and high-privilege prompt signing [17] help contain the blast radius of successful injections and reduce the risk of privilege escalation.

V. CONCLUSION

As demonstrated through real-world incidents ranging from prompt leakage to large-scale data exfiltration, attackers continue to exploit the inability of LLMs to reliably distinguish between benign context and adversarial instructions. The reviewed literature shows a clear evolution in both attack sophistication and defensive research, highlighting that traditional syntactic filters or isolated safeguards are no longer adequate. Instead, recent advances—including semantic detection frameworks, attention-pattern analysis, game-theoretic detectors, and multi-agent or ensemble-based systems—demonstrate significant progress in identifying and mitigating injection attempts across both direct and indirect vectors. At the same time, each detection category carries inherent limitations: syntactic filters can be easily bypassed through rephrasing, semantic models must balance false positives and interpretability, attention-based detectors require privileged access to model internals, and ensemble approaches introduce architectural complexity and computational overhead. These constraints reinforce the necessity of a holistic, multi-layered security strategy that integrates input validation, semantic reasoning, behavioral monitoring, architectural sandboxing, and continuous adversarial testing. As LLMs become increasingly embedded in critical applications—from code generation and retrieval-augmented decision systems to enterprise automation—the implications of prompt injection extend far beyond model misbehavior and into the domains of privacy, safety, and organizational risk. The findings reviewed in this work underscore that securing LLMs is not solely a technical challenge but an ongoing operational responsibility requiring continuous updates, evolving threat libraries, and systematic evaluation. Ultimately, effective protection against prompt injection will depend on the interplay between improved model architectures, robust detection frameworks, and strong system-level controls. By adopting defense-in-depth principles and integrating the best practices outlined in this study, developers and organizations can better safeguard LLM-driven systems against adversarial manipulation and ensure safer, more trustworthy deployments of large-scale AI technologies.

ACKNOWLEDGMENT

This research project is funded by the University of Technology and Applied Sciences, Oman, through the Internal Research Funding Program grant No IRFP-25-39.

REFERENCES

- [1] S. R. Dubey and H. S. Lamkuche, "Breaking the Shield: Adversarial Jailbreak Attacks and Defense Mechanisms in Large Language Models," in *2025 International Conference on Emerging Information Technology and Engineering Solutions (EITES)*, Jul. 2025, no. 2, pp. 82–86, doi: 10.1109/EITES66543.2025.00023.
- [2] M. Q. Li and B. C. M. Fung, "Security concerns for Large Language Models: A survey," *J. Inf. Secur. Appl.*, vol. 95, p. 104284, Dec. 2025, doi: 10.1016/j.jisa.2025.104284.
- [3] C. Chen, W. Qu, S. Su, Y. Feng, and T. Li, "A comprehensive review of LLM-based content moderation: advancements, challenges, and future directions," *Knowledge-Based Syst.*, vol. 330, p. 114689, Nov. 2025, doi: 10.1016/j.knosys.2025.114689.
- [4] S. Alla and A. S. Sichani, "Cyberattacks on Large Language Models - Attack Detection and Architecture Adaptability," in *Conference Proceedings - IEEE SOUTHEASTCON*, Mar. 2025, pp. 143–148, doi: 10.1109/SoutheastCon56624.2025.10971722.
- [5] M. Kim, T. Kwon, K. Shim, and B. Kim, "Protection of LLM Environment Using Prompt Security," in *2024 15th International Conference on Information and Communication Technology Convergence (ICTC)*, Oct. 2024, pp. 1715–1719, doi: 10.1109/ICTC62082.2024.10827351.
- [6] P. Kulkarni and A. Namer, "Temporal Context Awareness: A Defense Framework Against Multi-Turn Manipulation Attacks on Large Language Models," in *2025 IEEE Conference on Artificial Intelligence (CAI)*, May 2025, pp. 930–935, doi: 10.1109/CAI64502.2025.00164.
- [7] M. Chernyshev, Z. Baig, and R. Doss, "Forensic Analysis of Indirect Prompt Injection Attacks on LLM Agents," in *Proceedings - 2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications, TPS-ISA 2024*, 2024, pp. 409–411, doi: 10.1109/TPS-ISA62245.2024.00053.
- [8] A. Urman and M. Makhortkyh, "The silence of the LLMs: Cross-lingual analysis of guardrail-related political bias and false information prevalence in ChatGPT, Google Bard (Gemini), and Bing Chat," *Telemat. Informatics*, vol. 96, p. 102211, Jan. 2025, doi: 10.1016/j.tele.2024.102211.
- [9] L. Beurer-Kellner *et al.*, "Design Patterns for Securing LLM Agents against Prompt Injections," Jun. 2025, [Online]. Available: <http://arxiv.org/abs/2506.08837>.
- [10] K. Zhou *et al.*, "The Hidden Risks of Large Reasoning Models: A Safety Assessment of R1," Feb. 2025, [Online]. Available: <http://arxiv.org/abs/2502.12659>.
- [11] X. Chen, M. Zhu, and X. Shang, "Attack Detection Based on Domain-Specific Large Language Models," in *2025 5th International Conference on Neural Networks, Information and Communication Engineering, NNICE 2025*, 2025, pp. 1190–1193, doi: 10.1109/NNICE64954.2025.11064357.
- [12] Z. Wang, N. Nagaraja, L. Zhang, H. Bahsi, P. Patil, and P. Liu, "To Protect the LLM Agent Against the Prompt Injection Attack with Polymorphic Prompt," in *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*, Jun. 2025, pp. 22–28, doi: 10.1109/DSN-S65789.2025.00037.
- [13] M. Esugo, O. Alao, and H. Mahmoud, "SHIELD: Security against Harmful Prompt Injection Evaluation and Language Detection Leveraging Ensemble Approach," in *IECON 2025 - 51st Annual Conference of the IEEE Industrial Electronics Society*, Oct. 2025, pp. 1–6, doi: 10.1109/IECON58223.2025.11221566.
- [14] L. A. Tien and P. Van Huong, "Optimizing Transformer Models for

- Prompt Jailbreak Attack Detection in AI Assistant Systems,” in *1st International Conference on Cryptography and Information Security, VCRIS 2024 - Proceedings*, 2024, pp. 1–4, doi: 10.1109/VCRIS63677.2024.10813380.
- [15] S. Omri, M. Abdelkader, and M. Hamdi, “MalPID: Malicious Prompt Injection Detection Dataset for Large Language Model based Applications,” in *2024 IEEE 11th International Conference on Communications and Networking, ComNet 2024 - Proceedings*, 2024, pp. 1–5, doi: 10.1109/ComNet64071.2024.10987374.
- [16] O. Muliarevych, “Enhancing System Security: LLM-Driven Defense Against Prompt Injection Vulnerabilities,” in *Conference Proceedings - 2024 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering, TCSET 2024*, Oct. 2024, pp. 420–423, doi: 10.1109/TCSET64720.2024.10755823.
- [17] T. Joshi, V. Naik, I. Mistry, and R. Mangrulkar, “Prevention of Prompt Injection Attacks Over Financial Applications Integrated with LLM,” in *Proceedings - 3rd International Conference on Advancement in Computation and Computer Technologies, InCACCT 2025*, Apr. 2025, pp. 225–230, doi: 10.1109/InCACCT65424.2025.11011372.
- [18] B. ElSaify and M. Baderelden, “Adversarial and Multilingual Threats in Retrieval-Augmented Generation: From Prompt Injection to Model Exploitation,” in *2025 2nd International Generative AI and Computational Language Modelling Conference (GACLM)*, Aug. 2025, pp. 155–162, doi: 10.1109/GACLM67198.2025.11231998.
- [19] Y. Ji, R. Li, and B. Mao, “Detection Method for Prompt Injection by Integrating Pre-trained Model and Heuristic Feature Engineering,” Jun. 2025, [Online]. Available: <http://arxiv.org/abs/2506.06384>.
- [20] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, “DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks,” in *Proceedings - IEEE Symposium on Security and Privacy*, May 2025, pp. 2190–2208, doi: 10.1109/SP61157.2025.00250.
- [21] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” 2024.
- [22] Y. Zhao and X. Li, “A Novel Security Framework against Prompt Injection Attacks,” in *2025 IEEE 5th International Conference on Computer Communication and Artificial Intelligence (CCAI)*, May 2025, pp. 509–514, doi: 10.1109/CCAI65422.2025.11189427.
- [23] K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen, “Attention Tracker: Detecting Prompt Injection Attacks in LLMs,” in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 2309–2322, doi: 10.18653/v1/2025.findings-naacl.123.
- [24] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: Decoding-enhanced BERT with Disentangled Attention,” Oct. 2021, [Online]. Available: <http://arxiv.org/abs/2006.03654>.
- [25] K. Ren, T. Zheng, Z. Qin, and X. Liu, “Adversarial Attacks and Defenses in Deep Learning,” *Engineering*, vol. 6, no. 3, pp. 346–360, 2020, doi: 10.1016/j.eng.2019.12.012.
- [26] Q. Lan *et al.*, “Hybrid Constitutional Classifiers for Prompt Injection Defense,” in *2025 IEEE International Conference on Electro Information Technology (eIT)*, May 2025, pp. 225–229, doi: 10.1109/eIT64391.2025.11103606.